

CITYCARE HOSPITAL MANAGEMENT SYSTEM

WEEK 3: DOCTOR AND PHARMACY INVENTORY MANAGEMENT SYSTEM

1. Scenario

CityCare Hospital manages doctors and medicines used for patient treatment. The system should maintain doctor details, track medicines prescribed or supplied by doctors, and monitor medicine inventory levels to ensure medicines are available when required.

The system contains:

- Doctor details
- Medicine details
- Pharmacy inventory details
- Relationships between doctors and medicines
- Stock availability information
- Inventory reports

2. Create Doctor and Pharmacy Inventory Tables

Doctor Table

The Doctor table stores information about doctors working in CityCare Hospital.

Attributes

Attribute	Data Type	Constraint
Doctor_ID	NUMBER	Primary Key
Doctor_Name	VARCHAR2(100)	NOT NULL
Specialization	VARCHAR2(100)	NOT NULL
Phone_Number	VARCHAR2(15)	UNIQUE
Salary	NUMBER(10,2)	CHECK
Department_ID	NUMBER	Foreign Key

```

SQL> CREATE TABLE Doctor (
2     Doctor_ID NUMBER PRIMARY KEY,
3     Doctor_Name VARCHAR2(100) NOT NULL,
4     Specialization VARCHAR2(100) NOT NULL,
5     Phone_Number VARCHAR2(15) UNIQUE,
6     Salary NUMBER(10,2) CHECK (Salary > 0),
7     Department_ID NUMBER,
8     CONSTRAINT fk_doctor_department
9         FOREIGN KEY (Department_ID)
10        REFERENCES Department(Department_ID)
11 );

```

Table created.

3. Pharmacy Inventory Table

The Pharmacy_Inventory table stores information about medicines available in the hospital pharmacy.

Attributes

Attribute	Data Type	Constraint
Inventory_ID	NUMBER	Primary Key
Medicine_ID	NUMBER	Foreign Key
Medicine_Name	VARCHAR2(100)	NOT NULL
Stock_Quantity	NUMBER	CHECK
Unit_Price	NUMBER(10,2)	CHECK

SQL Query

```
SQL> CREATE TABLE Pharmacy_Inventory (  
2     Inventory_ID NUMBER PRIMARY KEY,  
3     Medicine_ID NUMBER UNIQUE,  
4     Medicine_Name VARCHAR2(100) NOT NULL,  
5     Stock_Quantity NUMBER CHECK (Stock_Quantity >= 0),  
6     Unit_Price NUMBER(10,2) CHECK (Unit_Price > 0),  
7     CONSTRAINT fk_inventory_medicine  
8         FOREIGN KEY (Medicine_ID)  
9         REFERENCES Medicine(Medicine_ID)  
10 );
```

Table created.

```
SQL> DESC Pharmacy_Inventory;
```

Name	Null?	Type
INVENTORY_ID	NOT NULL	NUMBER
MEDICINE_ID		NUMBER
MEDICINE_NAME	NOT NULL	VARCHAR2(100)
STOCK_QUANTITY		NUMBER
UNIT_PRICE		NUMBER(10,2)

4. Establish Relationships

Doctor and Medicine

One doctor can prescribe multiple medicines.

One medicine can be prescribed by multiple doctors.

Relationship:

Doctor → Medicine is Many-to-Many

This relationship can be implemented using a separate table such as Doctor_Medicine.

Medicine and Pharmacy Inventory

Each medicine has one inventory record.

Each inventory record belongs to one medicine.

Relationship:

Medicine → Pharmacy Inventory is One-to-One

The Medicine_ID in Pharmacy_Inventory is UNIQUE so that one medicine can have only one inventory record.

5. Primary Keys

Primary keys are used to uniquely identify records.

Doctor Table

Doctor_ID → Primary Key

Pharmacy_Inventory Table

Inventory_ID → Primary Key

6. Foreign Key

The Pharmacy_Inventory table contains Medicine_ID as a foreign key.

```
CONSTRAINT fk_inventory_medicine  
    FOREIGN KEY (Medicine_ID)  
    REFERENCES Medicine(Medicine_ID);
```

This connects the Pharmacy_Inventory table with the Medicine table.

7. Constraints

Unique Medicine ID

Medicine_ID is made UNIQUE in the Pharmacy_Inventory table.

```
Medicine_ID NUMBER UNIQUE
```

This ensures that the same medicine cannot have multiple inventory records.

NOT NULL Constraint

Medicine_Name cannot be empty.

```
Medicine_Name VARCHAR2(100) NOT NULL
```

CHECK Constraint for Stock Quantity

Stock quantity cannot be negative.

```
Stock_Quantity NUMBER CHECK (Stock_Quantity >= 0)
```

CHECK Constraint for Unit Price

Unit price must be greater than zero.

```
Unit_Price NUMBER(10,2) CHECK (Unit_Price > 0)
```

8. Insert Values into Pharmacy Inventory Table

```
SQL> INSERT INTO Medicine VALUES
  2  (201, 'Paracetamol', 'Pain Relief', 2.50);

1 row created.

SQL>
SQL> INSERT INTO Medicine VALUES
  2  (202, 'Amoxicillin', 'Antibiotic', 8.00);

1 row created.

SQL>
SQL> INSERT INTO Medicine VALUES
  2  (203, 'Cetirizine', 'Allergy', 3.50);

1 row created.

SQL>
SQL> INSERT INTO Medicine VALUES
  2  (204, 'Omeprazole', 'Gastric', 5.00);

1 row created.

SQL>
SQL> INSERT INTO Medicine VALUES
  2  (205, 'Azithromycin', 'Antibiotic', 12.00);

1 row created.

SQL>
SQL> INSERT INTO Medicine VALUES
  2  (206, 'Ibuprofen', 'Pain Relief', 4.00);

1 row created.

SQL>
SQL> COMMIT;
```

```
SQL>
SQL> SELECT * FROM Medicine;
```

```
MEDICINE_ID
-----
MEDICINE_NAME
```

```
-----
CATEGORY                                     UNIT_PRICE
-----
201
Paracetamol
Pain Relief                                     2.5
```

```
202
Amoxicillin
Antibiotic                                     8
```

```
MEDICINE_ID
-----
MEDICINE_NAME
```

```
-----
CATEGORY                                     UNIT_PRICE
-----
203
Cetirizine
Allergy                                     3.5
```

```
204
Omeprazole
```

```
MEDICINE_ID
-----
MEDICINE_NAME
```

```
-----
CATEGORY                                     UNIT_PRICE
-----
Gastric                                     5
```

```
205
```

```

      203
Cetirizine
Allergy
      3.5

      204
Omeprazole

MEDICINE_ID
-----
MEDICINE_NAME
-----
CATEGORY                                UNIT_PRICE
-----
Gastric                                5

      205
Azithromycin
Antibiotic
      12

      206
MEDICINE_ID
-----
MEDICINE_NAME
-----
CATEGORY                                UNIT_PRICE
-----
Ibuprofen
Pain Relief
      4

6 rows selected.

SQL> |

```

9. Insert Values into Doctor Table

```

SQL> CREATE TABLE Doctor (
  2     Doctor_ID NUMBER PRIMARY KEY,
  3     Doctor_Name VARCHAR2(100) NOT NULL,
  4     Specialization VARCHAR2(100) NOT NULL,
  5     Phone_Number VARCHAR2(15) UNIQUE,
  6     Salary NUMBER(10,2) CHECK (Salary > 0),
  7     Department_ID NUMBER,
  8     CONSTRAINT fk_doctor_department
  9         FOREIGN KEY (Department_ID)
 10         REFERENCES Department(Department_ID)
 11 );
CREATE TABLE Doctor (
      *

```

```
SQL> SELECT * FROM Doctor;
```

```
DOCTOR_ID
-----
DOCTOR_NAME
-----
SPECIALIZATION
-----
PHONE_NUMBER      SALARY DEPARTMENT_ID
-----
101
Dr. Arun Kumar
Cardiologist
9876543210      120000      1
```

```
DOCTOR_ID
-----
DOCTOR_NAME
-----
SPECIALIZATION
-----
PHONE_NUMBER      SALARY DEPARTMENT_ID
-----
102
Dr. Priya Sharma
Cardiologist
9876543211      115000      1
```

```
DOCTOR_ID
-----
DOCTOR_NAME
-----
SPECIALIZATION
-----
PHONE_NUMBER      SALARY DEPARTMENT_ID
-----
103
```

```
SQL>
SQL> INSERT INTO Doctor VALUES
  2 (102, 'Dr. Priya Sharma', 'Cardiologist', '9876543211', 115000, 1);

1 row created.
```

```
SQL>
SQL> INSERT INTO Doctor VALUES
  2 (103, 'Dr. Rahul Kumar', 'Neurologist', '9876543212', 110000, 2);

1 row created.
```

```
SQL>
SQL> INSERT INTO Doctor VALUES
  2 (104, 'Dr. Meena Raj', 'Orthopedic Surgeon', '9876543213', 125000, 3);

1 row created.
```

```
SQL>
SQL> INSERT INTO Doctor VALUES
  2 (105, 'Dr. Kavitha Devi', 'Pediatrician', '9876543214', 95000, 4);

1 row created.
```

```
SQL>
SQL> COMMIT;
```

```
Commit complete.
```

```
SQL>
SQL> SELECT * FROM Doctor;
```

```
DOCTOR_ID
-----
DOCTOR_NAME
-----
SPECIALIZATION
-----
```

DOCTOR_NAME		
SPECIALIZATION		
PHONE_NUMBER	SALARY	DEPARTMENT_ID
103		
Dr. Rahul Kumar		
Neurologist		
9876543212	110000	2
DOCTOR_ID		
DOCTOR_NAME		
SPECIALIZATION		
PHONE_NUMBER	SALARY	DEPARTMENT_ID
104		
Dr. Meena Raj		
Orthopedic Surgeon		
9876543213	125000	3
DOCTOR_ID		
DOCTOR_NAME		
SPECIALIZATION		
PHONE_NUMBER	SALARY	DEPARTMENT_ID
105		
Dr. Kavitha Devi		
Pediatrician		
9876543214	95000	4

10. CRUD Operations

Insert

Add a new medicine to the pharmacy inventory

```
SQL> INSERT INTO Medicine
  2  VALUES (207, 'Vitamin C', 'Supplement', 6.00);

1 row created.

SQL>
SQL> COMMIT;

Commit complete.

SQL>
SQL> SELECT * FROM Medicine
  2  WHERE Medicine_ID = 207;

MEDICINE_ID
-----
MEDICINE_NAME
-----
CATEGORY                                           UNIT_PRICE
-----
207
Vitamin C
Supplement                                           6
```

Update

Update the stock quantity of Vitamin C.

```
SQL> UPDATE Medicine
  2  SET Unit_Price = 7.00
  3  WHERE Medicine_ID = 207;
```

1 row updated.

```
SQL>
SQL> COMMIT;
```

Commit complete.

```
SQL>
SQL> SELECT * FROM Medicine
  2  WHERE Medicine_ID = 207;
```

MEDICINE_ID	MEDICINE_NAME	CATEGORY	UNIT_PRICE
207	Vitamin C	Supplement	7

Delete

Delete the Vitamin C inventory record.

```
SQL> DELETE FROM Medicine
  2  WHERE Medicine_ID = 207;
```

1 row deleted.

```
SQL>
SQL> COMMIT;
```

Commit complete.

```
SQL>
SQL> SELECT * FROM Medicine
  2  WHERE Medicine_ID = 207;
```

no rows selected

```
SQL> |
```

11. Report 1: Doctor-wise Medicine Report

Display:

- Doctor name
- Number of medicines
- Available stock

This report provides information about doctors and medicine inventory.

```
SQL> SELECT
  2     d.Doctor_Name,
  3     COUNT(dm.Medicine_ID) AS Medicine_Count,
  4     NVL(SUM(pi.Stock_Quantity), 0) AS Available_Stock
  5 FROM Doctor d
  6 LEFT JOIN Doctor_Medicine dm
  7     ON d.Doctor_ID = dm.Doctor_ID
  8 LEFT JOIN Pharmacy_Inventory pi
  9     ON dm.Medicine_ID = pi.Medicine_ID
 10 GROUP BY d.Doctor_Name
 11 ORDER BY d.Doctor_Name;
```

DOCTOR_NAME

MEDICINE_COUNT	AVAILABLE_STOCK
Dr. Arun Kumar	2
	750

Dr. Kavitha Devi	1
	300

Dr. Meena Raj	1
	0

DOCTOR_NAME

MEDICINE_COUNT	AVAILABLE_STOCK
Dr. Priya Sharma	2
	800

Dr. Rahul Kumar	2
	250

12. Report 2: Stock Availability Report

Display:

- Medicine name
- Stock quantity
- Stock status

```
SQL> SELECT
  2     Medicine_Name,
  3     Stock_Quantity,
  4     CASE
  5         WHEN Stock_Quantity = 0 THEN 'OUT OF STOCK'
  6         WHEN Stock_Quantity <= 100 THEN 'LOW STOCK'
  7         ELSE 'AVAILABLE'
  8     END AS Stock_Status
  9 FROM Pharmacy_Inventory
 10 ORDER BY Medicine_Name;
```

```
MEDICINE_NAME
-----
STOCK_QUANTITY STOCK_STATUS
-----
Amoxicillin
          250 AVAILABLE
```

```
Azithromycin
          100 LOW STOCK
```

```
Cetirizine
          300 AVAILABLE
```

```
MEDICINE_NAME
-----
STOCK_QUANTITY STOCK_STATUS
-----
Ibuprofen
          0 OUT OF STOCK
```

```
Omeprazole
          150 AVAILABLE
```

```
Paracetamol
          500 AVAILABLE
```

```
6 rows selected.
```

13. Report 3: Inventory Analysis

13.1 Total Medicines Available

```
SQL> SELECT COUNT(*) AS Total_Medicines
2  FROM Pharmacy_Inventory;

TOTAL_MEDICINES
-----
6
```

13.2 Medicines Out of Stock

```
SQL> SELECT COUNT(*) AS Out_of_Stock_Medicines
2  FROM Pharmacy_Inventory
3  WHERE Stock_Quantity = 0;

OUT_OF_STOCK_MEDICINES
-----
1
```

13.3 Highest Stocked Medicine

```
SQL> SELECT Medicine_Name, Stock_Quantity
2  FROM Pharmacy_Inventory
3  WHERE Stock_Quantity = (
4      SELECT MAX(Stock_Quantity)
5      FROM Pharmacy_Inventory
6  );

MEDICINE_NAME
-----
STOCK_QUANTITY
-----
Paracetamol
500
```

13.4 Average Inventory Quantity

```
SQL> SELECT ROUND(AVG(Stock_Quantity), 2) AS Average_Inventory
2  FROM Pharmacy_Inventory;

AVERAGE_INVENTORY
-----
216.67
```

14. Summary

In Week 3, the CityCare Hospital Management System was extended to manage doctors and pharmacy inventory.

The Doctor table stores doctor information, while the Pharmacy_Inventory table stores medicine stock information.

Primary keys, foreign keys, UNIQUE, NOT NULL, and CHECK constraints were implemented to maintain data integrity.

CRUD operations were performed on the pharmacy inventory, and reports were generated to analyse medicine availability and inventory levels.

15. Conclusion

The Week 3 implementation provides an effective database structure for managing doctors and pharmacy inventory in CityCare Hospital.

The system helps monitor medicine stock, identify unavailable medicines, and generate useful inventory reports. This improves the organization and management of hospital pharmacy data.